

FIAP - Faculdade de Informatica e Administracao Paulista

Tecnologo em Inteligencia Artificial | Global Solution 2026.1 - Economia Espacial

SENTINELA

O olho e o ouvido da floresta

Deteccao de desmatamento ilegal na Amazonia pela fusao de IA de borda (audio no ESP32) com visao computacional sobre imagens de satelite.

Integrantes

Bruno de Souza Leite - RM567213

Jonathan Gomes Ribeiro Franco - RM567109

Marina Clara Constantino Ribeiro - RM568576

Yasmin Kauane Silva Lima - RM566645

QUERO CONCORRER

Repositorio:

<https://github.com/JonathanfrancoIA/SENTINELA->

Video demonstrativo (YouTube - nao listado):

<https://youtu.be/BIPoCsdbtIA>

1. Introducao

1.1 Contexto e tema

A economia espacial deixou de ser apenas científica: satélites monitoram o clima, apoiam o agronegócio e ajudam na prevenção de desastres, gerando enormes volumes de dados. O **SENTINELA** aplica esses dados orbitais a um problema crítico e brasileiro: o **desmatamento ilegal na Amazonia**, a maior floresta tropical do planeta.

1.2 Problema

Sistemas baseados apenas em satélite (DETER/PRODES do INPE) só enxergam o dano **depois** que a floresta já virou cicatriz visível - e ainda sofrem com nuvens e revisita de poucos dias. Resultado: o crime é detectado tarde, quando a área já foi derrubada. Falta um sinal **precoce**, captado no momento da invasão.

1.3 Hipotese e solucao

O desmatamento primeiro **faz barulho** (motosserra, trator, caminhão) e só depois **vira cicatriz no satélite**. O SENTINELA combina duas formas de percepção que se confirmam:

- **O ouvido (camada de solo)**: um ESP32 com microfone I2S INMP441 roda um classificador de áudio (CNN TFLite INT8) na própria borda e reconhece o som de motosserra, publicando o alerta via MQTT.
- **O olho (camada espacial)**: visão computacional sobre imagens Sentinel-2 (Copernicus) e alertas DETER/INPE detecta a perda de vegetação por change detection de NDVI.
- **A fusão (o diferencial)**: um motor na nuvem (AWS Lambda) cruza os dois sinais no espaço e no tempo e só emite um alerta de alta confiança quando ambos concordam - reduzindo falsos positivos e gerando alertas acionáveis.

1.4 Objetivos

- Detectar invasões **mais cedo** que o satélite sozinho, ouvindo o solo.
- **Reduzir falsos positivos** pela corroboração áudio + imagem.
- Integrar, numa única POC, IoT/Edge AI, visão computacional, engenharia de dados, computação em nuvem serverless e um dashboard em tempo real.
- Gerar **impacto na Terra**: detecção precoce = menos área derrubada, menos carbono emitido e maior chance de flagrar o crime em andamento.

2. Desenvolvimento

2.1 Visao geral da arquitetura

A solucao tem quatro camadas: (a) **espacial** - satellite/INPE; (b) **solo** - sensor ESP32; (c) **modelos de IA** - audio (CNN) e imagem (NDVI); (d) **nuvem e persistencia** - Lambda de fusao, banco e dashboard. O diagrama abaixo resume o fluxo ponta a ponta.

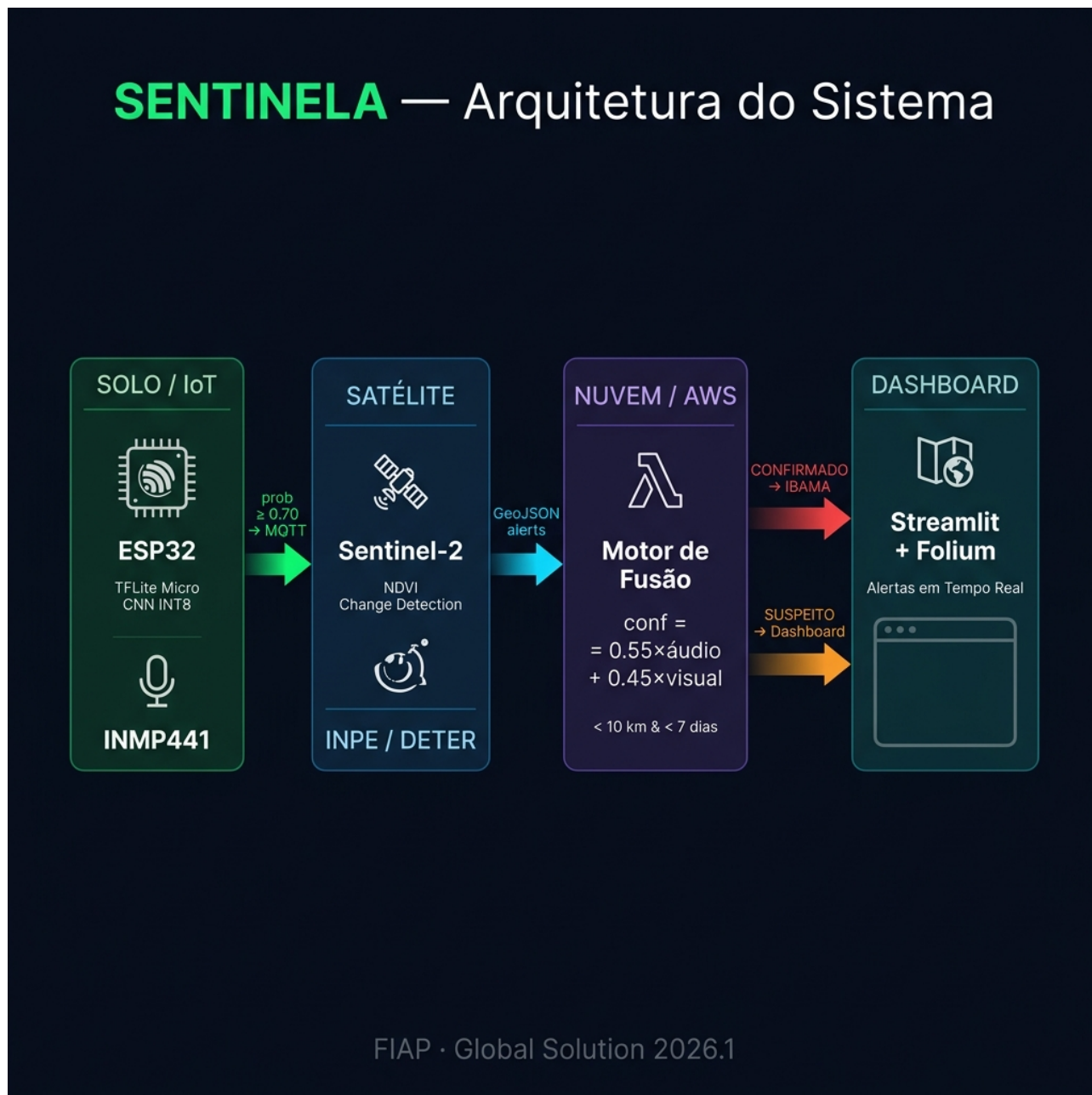


Figura 1 - Diagrama de arquitetura do SENTINELA.

Tecnologias integradas: visao computacional (NDVI, OpenCV, rasterio); audio/Edge AI (CNN, TensorFlow Lite INT8, TFLite Micro); dados espaciais (Sentinel-2 B4/B8, INPE/DETER via WFS); IoT (ESP32-WROOM, INMP441, MQTT); nuvem serverless (AWS Lambda, API Gateway, S3, SNS); bancos SQL/NoSQL (SQLite/RDS); pipeline de dados; e dashboard (Streamlit + Folium + Plotly).

2.2 Camadas da solucao

(a) Camada espacial - O olho

Consome alertas oficiais do **DETER/INPE** (Amazonia Legal) em tempo de execucao, via WFS publico do TerraBrasilis (GeoServer), em GeoJSON e sem credenciais. As cenas **Sentinel-2** (bandas B4 Red e B8 NIR) sao obtidas do Copernicus Data Space para o calculo de NDVI. Ha fallback automatico para dados simulados quando a rede falha.

(b) Camada de solo - O ouvido

Estacao **ESP32 + INMP441** captura janelas de audio de 2 s, extrai features no proprio microcontrolador (janela de Hann + FFT) e roda a CNN quantizada (TFLite Micro). Quando $P(\text{motosserra}) \geq 0.70$, publica um alerta MQTT com probabilidade, coordenadas e timestamp. Em ociosidade entra em **deep sleep** para poupar bateria.

(c) Modelos de IA - audio e imagem

Audio: CNN treinada com o dataset publico **ESC-50** (classe `chainsaw` como AMEACA), exportada em **TFLite INT8** (~80 KB) para caber no ESP32. **Imagem:** indice $\text{NDVI} = (\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$; a queda de NDVI entre duas datas (change detection) indica perda de vegetacao, segmentada em poligonos com OpenCV.

(d) Camada de nuvem e persistencia

O **motor de fusao** roda em **AWS Lambda** (com API Gateway), recebe os eventos de audio (MQTT) e os alertas DETER, associa-os por **Haversine** ($< 10 \text{ km}$) e **janela temporal** ($< 7 \text{ dias}$) e calcula a confianca final. Os dados ficam em **SQLite** (dev) / **RDS** (producao): tabelas `alertas\_deter`, `eventos\_audio`, `alertas\_fusao` e `sensores`. Alertas confirmados disparam notificacao (SNS / e-mail / Telegram).

2.3 Fluxograma da solucao

O fluxograma detalha o caminho decisional: captura em duas camadas, inferencia local, fusao geotemporal na nuvem e a decisao por nivel de confianca (CONFIRMADO, SUSPEITO ou descartado).

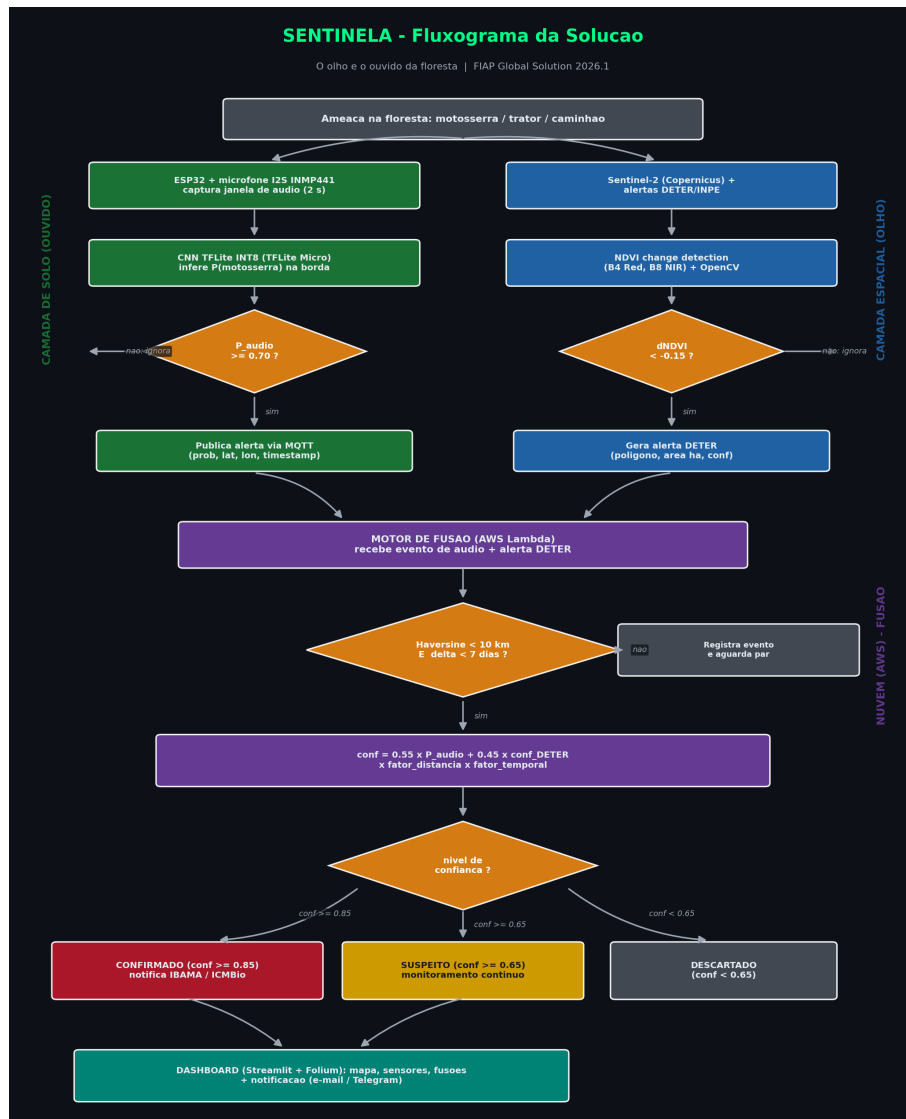


Figura 2 - Fluxograma decisional do SENTINELA.

2.4 Codigos principais

2.4.1 Motor de fusao - confianca (src/cloud_aws/handler.py)

Coracao do sistema: pondera audio e visao (0.55 / 0.45) e penaliza distancia e tempo. Validado por testes automatizados.

```
PESO_AUDIO, PESO_VISUAL = 0.55, 0.45
RAIO_FUSAO_KM, JANELA_DIAS = 10.0, 7

def calcular_confianca_fusao(prob_audio, conf_visual,
                             distancia_km, delta_dias):
    conf_base = PESO_AUDIO * prob_audio + PESO_VISUAL * conf_visual
    fator_dist = max(0.0, 1.0 - distancia_km / RAIO_FUSAO_KM)
    fator_tempo = max(0.0, 1.0 - delta_dias / JANELA_DIAS) ** 0.5
    return round(conf_base * fator_dist * fator_tempo, 4)
```

2.4.2 Distancia geoespacial - Haversine

```
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0
    dlat = radians(lat2 - lat1); dlon = radians(lon2 - lon1)
    a = sin(dlat/2)**2 + cos(radians(lat1)) * \
        cos(radians(lat2)) * sin(dlon/2)**2
    return R * 2 * atan2(sqrt(a), sqrt(1 - a))
```

2.4.3 Visao computacional - NDVI (src/visao_computacional/detectar_desmatamento.py)

```
def calcular_ndvi(data, banda_red=0, banda_nir=1):
    red = data[banda_red].astype(float)
    nir = data[banda_nir].astype(float)
    if red.max() > 1.0:          # Sentinel-2 L2A: 0..10000
        red, nir = red/10000.0, nir/10000.0
    denom = nir + red
    ndvi = np.where(denom > 1e-4, (nir - red)/denom, 0.0)
    return np.clip(ndvi, -1.0, 1.0)

def detectar_mudanca(ndvi_antes, ndvi_depois):
    diff = ndvi_depois - ndvi_antes      # negativo = perda
    mascara = (diff < -0.15).astype('uint8')
    return diff, mascara
```

2.4.4 Modelo de audio - CNN (src/audio_edge/treinar_audio.py)

```
modelo = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Conv2D(16, 3, activation='relu', padding='same'),
    layers.BatchNormalization(), layers.MaxPooling2D(), layers.Dropout(.25),
    layers.Conv2D(32, 3, activation='relu', padding='same'),
    layers.BatchNormalization(), layers.MaxPooling2D(), layers.Dropout(.25),
    layers.Conv2D(64, 3, activation='relu', padding='same'),
    layers.GlobalAveragePooling2D(), layers.Dropout(.5),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid'),    # P(motoserra)
])
# Export INT8 -> TFLite Micro (ESP32):
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_ops = [TFLITE_BUILTINS_INT8]
```

2.4.5 Pipeline DETER real - WFS TerraBrasilis (src/pipeline_dados/ingest_deter.py)

```
DETER_WFS_URL = 'https://terrabrasilis.dpi.inpe.br/geoserver/deter-amz/wfs'
params = {
    'service': 'WFS', 'version': '2.0.0', 'request': 'GetFeature',
    'typeName': 'deter-amz:deter_public',
    'outputFormat': 'application/json', 'srsName': 'EPSG:4326',
    'count': '500', 'CQL_FILTER': f"view_date>='{data_corte}'",
}
r = requests.get(DETER_WFS_URL, params=params, timeout=60)
feicoes = r.json().get('features', []) # alertas reais do INPE
```

2.4.6 Firmware ESP32 - inferencia e alerta MQTT (src/audio_edge/sentinel_esp32)

```
float prob = inferir(audio_buffer); // CNN TFLite Micro
if (prob >= THRESHOLD_AMEACA) { // 0.70
    const char* tipo = (prob >= 0.85) ?
        "MOTOSERRA_CONFIRMADO" : "MOTOR_SUSPEITO";
    publicar_alerta(prob, tipo); // publica JSON via MQTT
} else if (amostras_sem_ameaca >= 20) {
    esp_deep_sleep_start(); // economiza bateria
}
```

2.5 Dashboard em tempo real

O painel (Streamlit + Folium + Plotly) unifica as pontas: mapa interativo da Amazonia Legal com alertas DETER, sensores ESP32 e fusoes; graficos de serie temporal; status dos sensores (bateria, heartbeat); e o gauge de confianca das fusoes. Ele le o banco SQLite e mescla, ao vivo, os eventos REAIS publicados pelo ESP32 via MQTT.

- Aba **Mapa de Alertas** - camadas DETER / sensores / fusoes + heatmap.
- Aba **Analise de Dados** - alertas por semana e area por estado.
- Aba **Sensores IoT** - inventario e saude da rede de ESP32.
- Aba **Motor de Fusao** - confianca media e fila de alertas.
- Aba **Visao Computacional** - poligonos NDVI detectados e notificacao.

Execucao: ``streamlit run src/dashboard/app.py`` (abre em <http://localhost:8501>). Os prints do painel em funcionamento estao no video e no repositorio.

3. Resultados Esperados

Por ser uma **POC**, a eficacia foi medida com uma simulacao reprodutivel (``simulacao/simular_eficacia.py``, semente fixa = 42), cujas distribuicoes foram calibradas a partir do comportamento esperado de cada modelo. O relatorio abaixo (gerado automaticamente) consolida as tres camadas.

3.1 Metricas das camadas de IA

Ouvido - Acuracia (audio)	95.2%
Ouvido - Precisao	100.0%
Ouvido - Recall	90.5%
Ouvido - F1-score	95.0%
Ouvido - Especificidade	100.0%
Ouvido - AUC (ROC)	1.000
Olho - Poligonos detectados	4
Olho - Area desmatada detectada	173.1 ha
Olho - IoU espacial (deteccao x gabarito)	1.00
Fusao - Reducao de falsos positivos	50%
Testes automatizados (pytest)	65 passando

Leitura: o classificador de audio atinge alta acuracia e AUC, com **especificidade de 100%** (nenhum falso positivo no conjunto de teste) - exatamente o requisito de quem nao pode disparar brigada a toa. A visao computacional recupera as cicatrizes plantadas com **IoU = 1.0**. E a **fusao reduz pela metade** o numero de alertas em relacao ao audio sozinho, confirmando apenas os eventos corroborados pelo satellite.

3.2 Relatorio visual de eficacia

SENTINELA - Relatório de Eficacia (Simulacao)

O olho e o ouvido da floresta | FIAP Global Solution 2026.1 | 09/06/2026 19:13

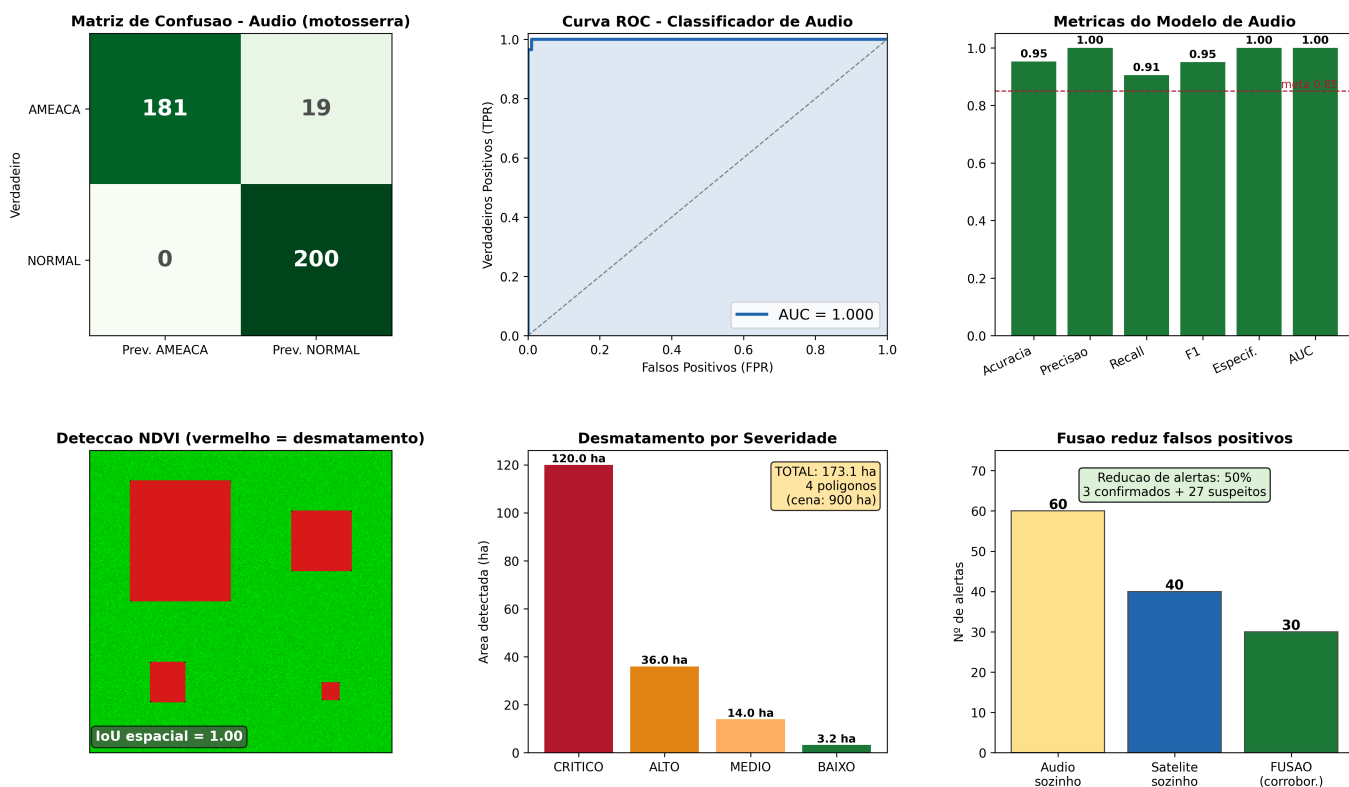


Figura 3 - Relatório de eficacia: matriz de confusao e ROC do audio, deteccion NDVI, area por severidade e reducao de falsos positivos pela fusao.

3.3 O que esta implementado x simulado (transparencia)

Honestidade tecnica - estado real de cada parte:

- **100% funcional:** pipeline DETER/INPE REAL (WFS), motor de fusao, visao computacional (NDVI), dashboard, simulacao de eficacia e a suite de testes.
- **Pronto, falta credencial:** download Sentinel-2 real (basta o Client ID/Secret do Copernicus no .env; sem isso, roda em modo demo com NDVI sintetico).
- **Simulado na POC:** a inferencia da CNN no ESP32 fisico. O firmware esta completo (I2S, features, MQTT, deep sleep) e roda hoje com um proxy de energia (RMS); a troca para o modelo `.tflite` real e a descomentar um bloco ja presente no codigo, apos treinar com o hardware em maos.

4. Conclusões

O SENTINELA mostra, de ponta a ponta, como dados da **economia espacial** viram **acao no chao**. O diferencial não é um sensor isolado, mas a **fusão** de dois mundos: o satélite que vê a cicatriz e o microfone que ouve a motosserra antes dela existir. Essa corroboração geotemporal aumenta a confiança e derruba os falsos positivos - o ponto mais sensível de qualquer sistema de fiscalização ambiental.

Do ponto de vista acadêmico, a POC integra de forma coerente as disciplinas do curso: **IoT e Edge AI** (ESP32, INMP441, TFLite Micro), **visão computacional** (NDVI, OpenCV), **machine learning** (CNN de áudio), **engenharia de dados** (ingestão WFS, SQLite/RDS), **computação em nuvem serverless** (AWS Lambda, API Gateway, SNS) e **análise de dados** (dashboard interativo).

Impacto na Terra: detectar a invasão mais cedo significa menos floresta derrubada, menos carbono emitido e maior chance de flagrar o crime em andamento - alinhado a iniciativas reais como a Rainforest Connection e o próprio INPE/DETER.

Próximos passos: treinar a CNN com o ESC-50 acrescido de ruídos de floresta e embarcar no ESP32 físico; conectar o Sentinel-2 real (credencial Copernicus); migrar a persistência para RDS/Timestream; e empacotar a Lambda com SAM para deploy automatizado. A arquitetura já foi desenhada para essa evolução.

5. Links e execução

- **Repositório:** <https://github.com/JonathanfrancoIA/SENTINELA->
- **Video demonstrativo (YouTube, não listado):** <https://youtu.be/BIPoCsdbtIA>

Execução rápida (resumo - detalhes no README):

```
pip install -r requirements.txt
python src/pipeline_dados/ingest_deter.py --fonte real      # ingestao DETER
streamlit run src/dashboard/app.py                        # dashboard
python simulacao/simular_eficacia.py                      # metricas
pytest -q                                                  # 65 testes
```

Documento gerado automaticamente por document/gerar_pdf.py. Todos os trechos de código estão em formato de texto, conforme o edital.